

De la GPU a GitHub

Entrenar con una AMD Radeon RX 6700 XT, gestionar entornos de Python y llevar tu código del portátil al laboratorio

The background of the slide is split diagonally from the bottom-left corner to the top-right corner. The upper-left portion is white, and the lower-right portion is a solid blue color. The text is positioned on the blue background.

GPU

ENTORNOS

CONDA

GITHUB

CPU vs GPU

Dos formas de cálculo

CPU (Unidad Central de Procesamiento)

Cerebro del ordenador

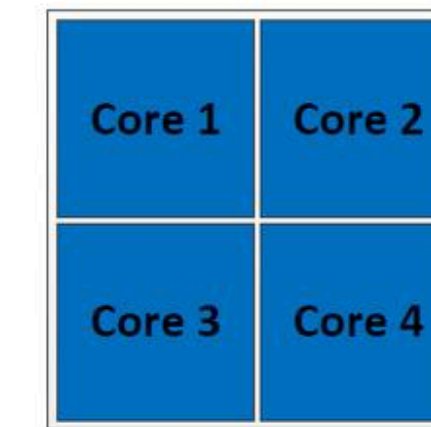
- Ejecuta el sistema y coordina todo.
- Pocos núcleos, muy sofisticados.

GPU (Unidad de Procesamiento Gráfico)

Muchísimas unidades de cálculo más sencillas que pueden trabajar en paralelo.

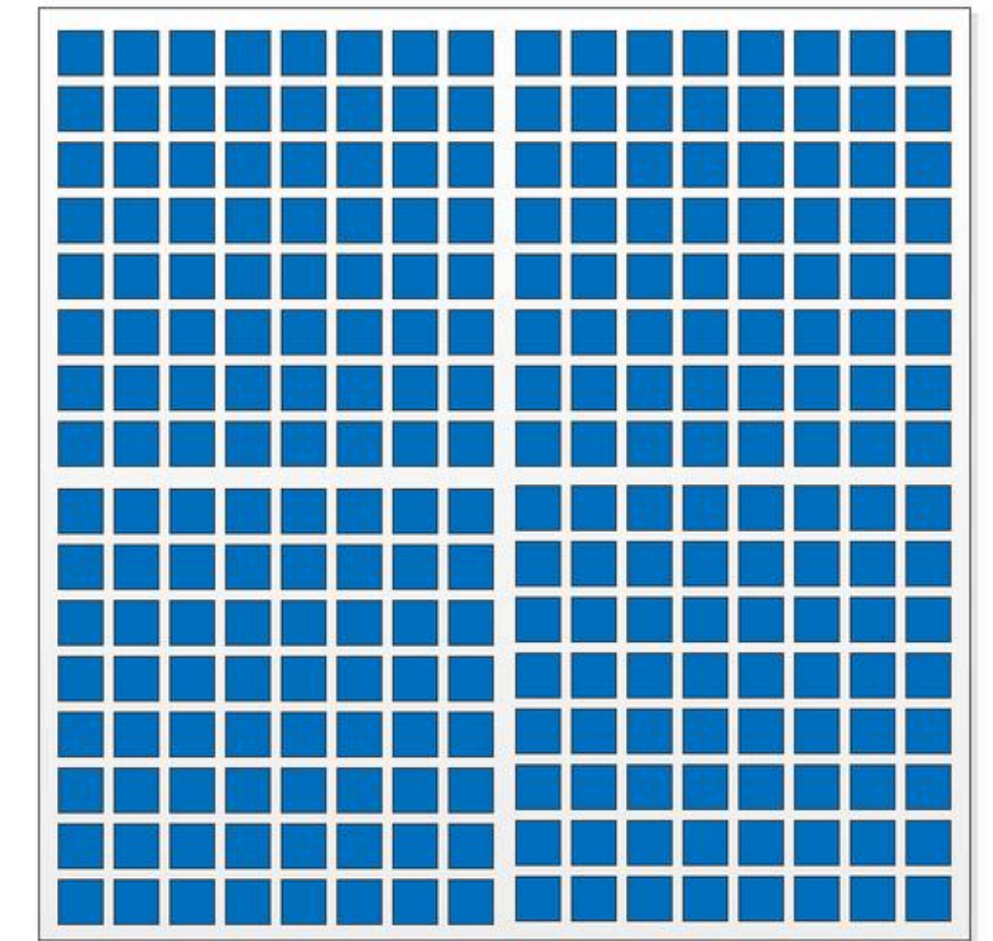
CPU

(Multiple cores)

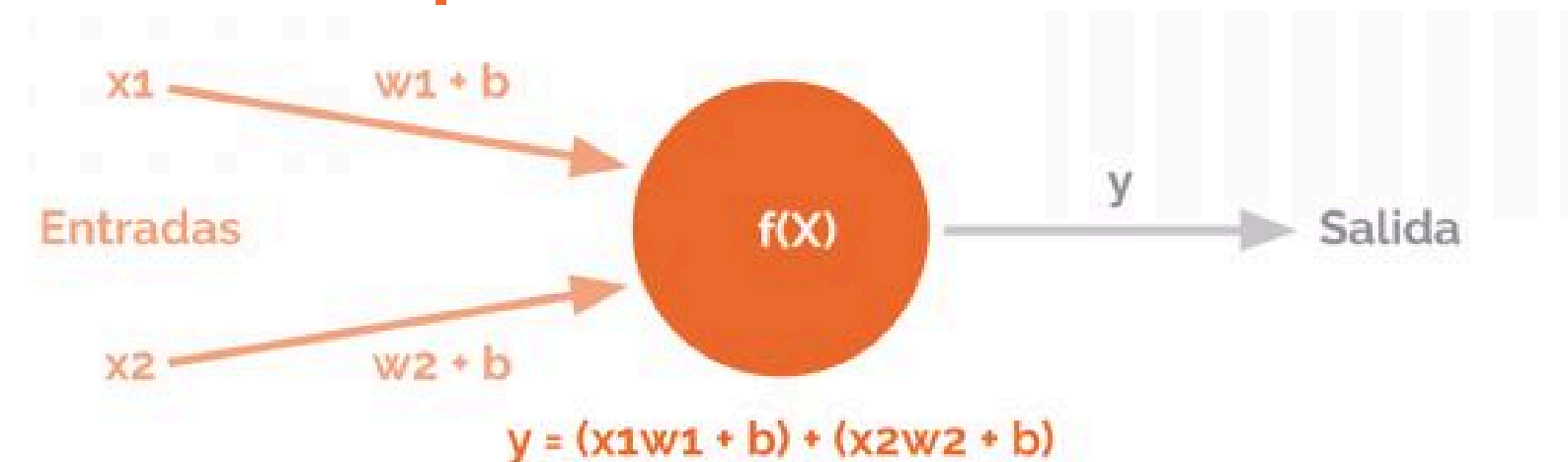


GPU

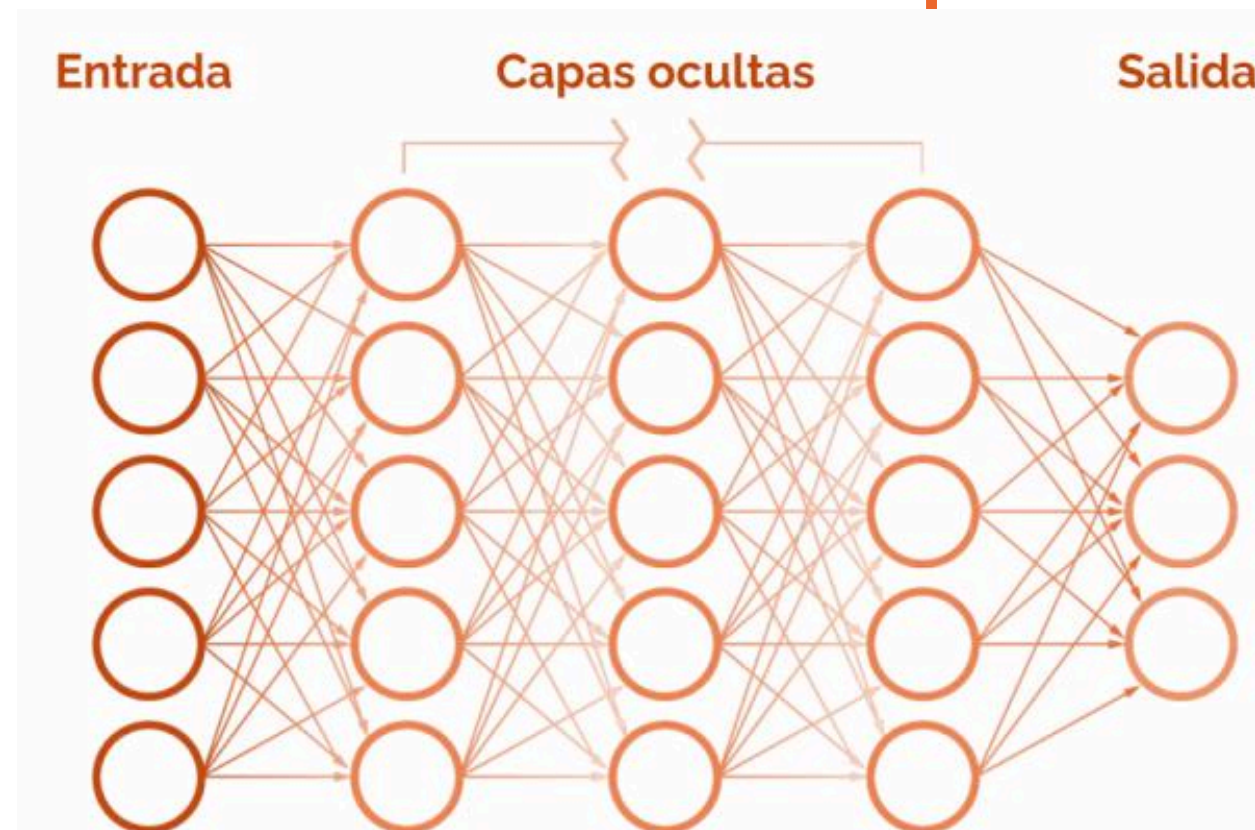
(Hundreds of cores)



Modelo simplificado de una neurona artificial



Red neuronal profunda con más de tres capas



CPU



GPU

AMD Radeon RX 6700 XT

Características Generales De La GPU

12 GB de VRAM

IMPORTANTE: Límite para el modelo + batch

40 CU (Compute Units)

2560 stream processors (unidades de cálculo)

≈ 13 TFLOPS

gfx1031

Identificador para ROCm

Más información: [AMD Radeon RX 6700 XT](#)



Instalación de PyTorch

Hay un entorno de PyTorch con GPU creado llamado **prolabs**.

```
# 1) La RX 6700 XT (gfx1031) se hace pasar por gfx1030, que sí está soportada
export HSA_OVERRIDE_GFX_VERSION=10.3.0
# 2) PyTorch para ROCm: copia la línea exacta del selector de pytorch.org
pip install torch torchvision --index-url https://download.pytorch.org/whl/rocmX.Y
```

```
import torch
print(torch.__version__)           # termina en +rocmX.Y
print(torch.version.hip)           # versión de HIP (None si no es ROCm)
print(torch.cuda.is_available())   # True: PyTorch ve la GPU
print(torch.cuda.get_device_name(0)) # AMD Radeon RX 6700 XT
```

Link de descarga oficial: <https://pytorch.org/get-started/locally/>

NOTE: Latest Stable PyTorch requires Python 3.10 or later.

PyTorch Build	Stable (2.14.0)	Preview (Nightly)			
Your OS	Linux	Mac	Windows		
Package	Pip	LibTorch	Source		
Language	Python	C++ / Java			
Compute Platform	CUDA 12.6	CUDA 13.0	CUDA 13.2	ROCm 7.14	CPU
Run this Command:	pip3 install torch torchvision --index-url https://download.pytorch.org/whl/rocm7.14				

Could not find the right platform for your hardware? See the [PyTorch Additional Platforms](#) page.

Estos ordenadores son de la universidad

No instales nada a nivel de sistema

Tus archivos locales pueden desaparecer

Lo que no esté en GitHub, no existe

Nunca dejes contraseñas ni tokens

GPU

ENTORNOS

CONDA

GITHUB

Entornos virtuales

Una caja de dependencias aislada para cada proyecto

El Python del sistema **no se debe de tocar**. Tenerlo en entornos virtuales separados hace que no haya conflictos entre dependencias, que sea reproducible en otro ordenador y desechable, es decir, en caso de que se rompa se borra y se crea uno nuevo.

Entorno: TFG

Scipy, Numpy 1.26

Entorno: Practica de IA

Torch 2.X, Numpy 2.0

Entorno: Otra asignatura

Pandas

Python ya viene con un entorno incluido:

Venv

- Dentro del proyecto: Se crea una carpeta con el nombre del entorno, pero nunca subir a Git
- No es portable: Un entorno de Windows, no sirve en Linux.

Nombres comunes de entorno: env, .env, .venv, venv

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install numpy torch
deactivate
```

Windows

```
python3 -m venv .venv
source .venv/bin/activate
pip install numpy torch
deactivate
```

Linux y MacOS

Requirements.txt

Gestión de las dependencias del proyecto

Archivo que contiene la lista de librerías y dependencias necesarias para que un proyecto de Python funcione correctamente. Para crearlo debemos de estar dentro del entorno.

Límites:

- Mezcla junto con dependencias.
- No fija la versión de Python necesaria.

```
# guardar lo instalado
pip freeze > requirements.txt
# recrearlo en otra máquina
pip install -r requirements.txt
```



A screenshot of a GitHub commit by user 'mgonzalz' titled 'feat: ejemplo de requirements'. The commit shows a file named 'requirements.txt' with 12 lines of code. The code lists various Python packages with their versions, separated by double equals signs. The packages listed are: contourpy, cyclor, fonttools, kiwisolver, matplotlib, numpy, packaging, pandas, pillow, pyparsing, python-dateutil, and six.

```
1  contourpy==1.4.0
2  cyclor==0.12.1
3  fonttools==4.65.0
4  kiwisolver==1.5.1
5  matplotlib==3.11.2
6  numpy==2.5.3
7  packaging==26.3
8  pandas==3.0.6
9  pillow==12.3.0
10 pyparsing==3.3.3
11 python-dateutil==2.9.0.post0
12 six==1.17.0
```

Para este ejemplo de requirements, tan solo se hizo 'pip install pandas matplotlib'

Poetry

Dependencias, entorno y versiones exactas en una sola herramienta.

Poetry resuelve esos problemas.

- **pyproject.toml** anota las librerías con rangos de versiones y la versión de Python.
- **poetry.lock** guarda la versión exacta de cada paquete que se instaló. Se debe de subir a Git para tener todos lo mismo.

Estos archivos se crean y actualizan solos al iniciar Poetry.



```
poetry init           # Inicializa el proyecto
poetry add pandas     # Añade una dependencia
poetry install        # Instala las dependencias
poetry run python train.py  # Ejecuta el script
```

base-github-template / pyproject.toml

mgonzalz refactor: añadir para que no cree un paquete de python y no salga el ...

Code Blame 21 lines (18 loc) · 487 Bytes

```
1  [project]
2  name = "base-github-template"
3  version = "0.1.0"
4  description = "Ejemplo de realización de archivo Poetry sin ver las dependencias."
5  authors = [
6      {name = "Maria González", email = "mgonzgarco@myuax.com"}
7  ]
8  readme = "README.md"
9  requires-python = ">=3.12"
10 dependencies = [
11     "pandas (>=3.0.6,<4.0.0)",
12     "matplotlib (>=3.11.2,<4.0.0)"
13 ]
14
15
16 [build-system]
17 requires = ["poetry-core>=2.0.0,<3.0.0"]
18 build-backend = "poetry.core.masonry.api"
19
20 [tool.poetry]
21 package-mode = false
```

Para poder utilizarlo se debe de seguir las instrucciones de descarga. Más información: <https://python-poetry.org/>

The background of the slide is split diagonally from the bottom-left corner to the top-right corner. The upper-left portion is white, and the lower-right portion is a solid blue color. The text is positioned on the blue background.

GPU

ENTORNOS

CONDA

GITHUB

Conda

Entornos con su propio Python y librerías que no son de Python

Gestor de entornos y paquetes que permite aislar Python, sus librerías y otras dependencias necesarias para cada proyecto.

- Elige la versión de Python: Cada entorno trae su propio intérprete
- Instala binarios no-Python: Compiladores, MKL, GDAL, R...
- Entornos globales: Viven en la carpeta de conda y se llaman por nombre



```
conda create -n prueba python=3.12
conda activate prueba
conda install pandas matplotlib # también puede hacerse con pip
# PyTorch se instala con pip, también en conda
pip install torch --index-url .../rocmX.Y

# compartir el entorno
conda env export > environment.yml
conda env create -f environment.yml
conda env list
```

Tipos de Conda

El comando **conda** es el mismo en los tres. Cambia lo que viene instalado de serie y de dónde se descargan los paquetes.

Más información: [Conda download](#)

Anaconda

conda, Python, cientos de paquetes científicos preinstalados y Anaconda Navigator, una interfaz gráfica.

Miniconda

Solo conda y Python. Instalas lo que necesitas en cada entorno: más ligero y más limpio.

Miniforge

Como Miniconda, pero usa el canal comunitario conda-forge por defecto, sin las condiciones del canal de Anaconda.

¿Qué herramienta elijo?

Una guía rápida según el tipo de proyecto

Sea cual sea: **un entorno por proyecto y el requirements.txt en el repositorio.**

Venv + pip

Prácticas y scripts sencillos, solo paquetes de Python. Viene de serie: nada que instalar.

Poetry

Proyectos que crecen, trabajo en equipo, TFG, o una librería que quieras publicar.

Conda

Necesitas librerías que no son de Python o varias versiones de Python en la misma máquina.





GPU

ENTORNOS

CONDA

GITHUB

Git y GitHub

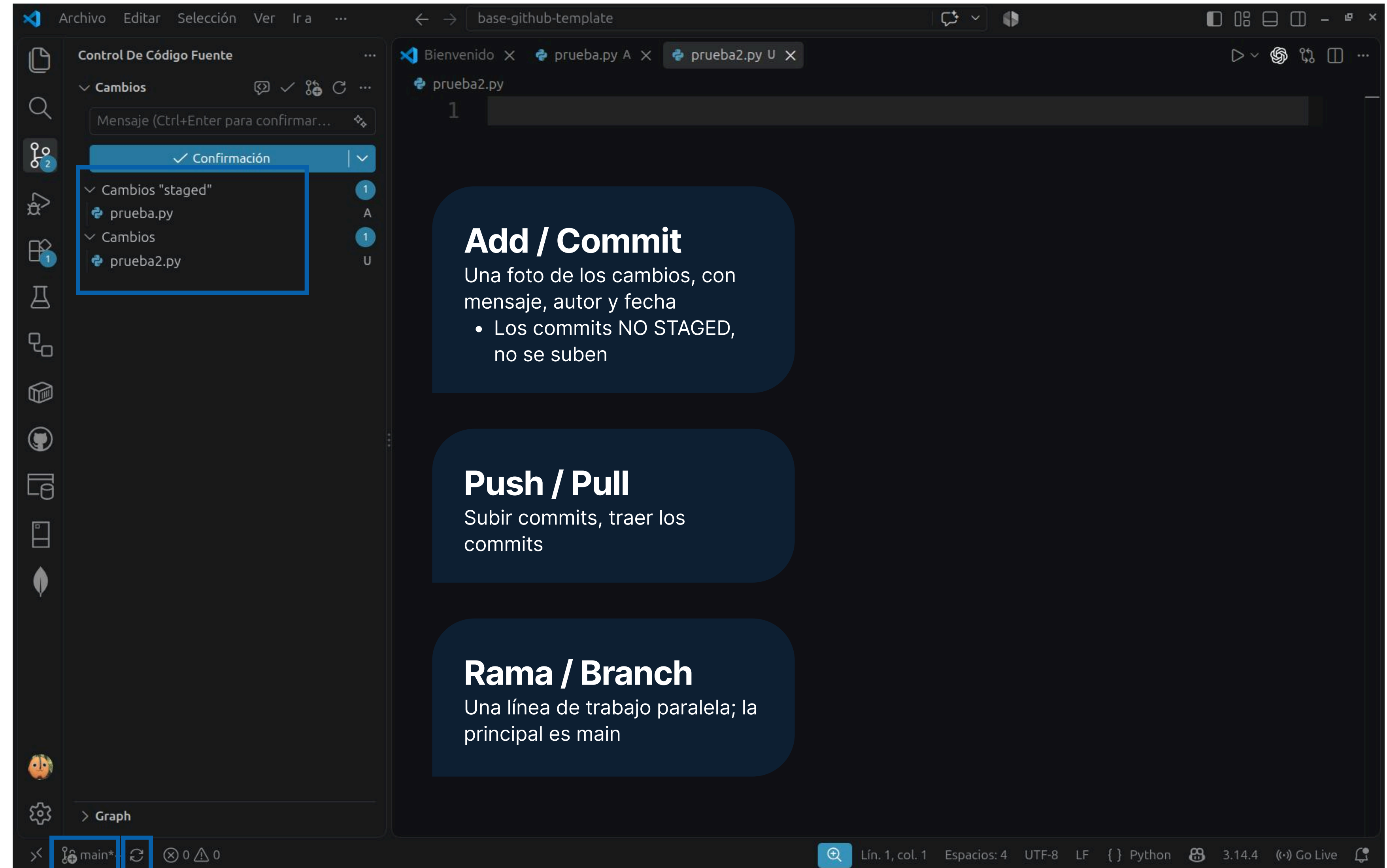
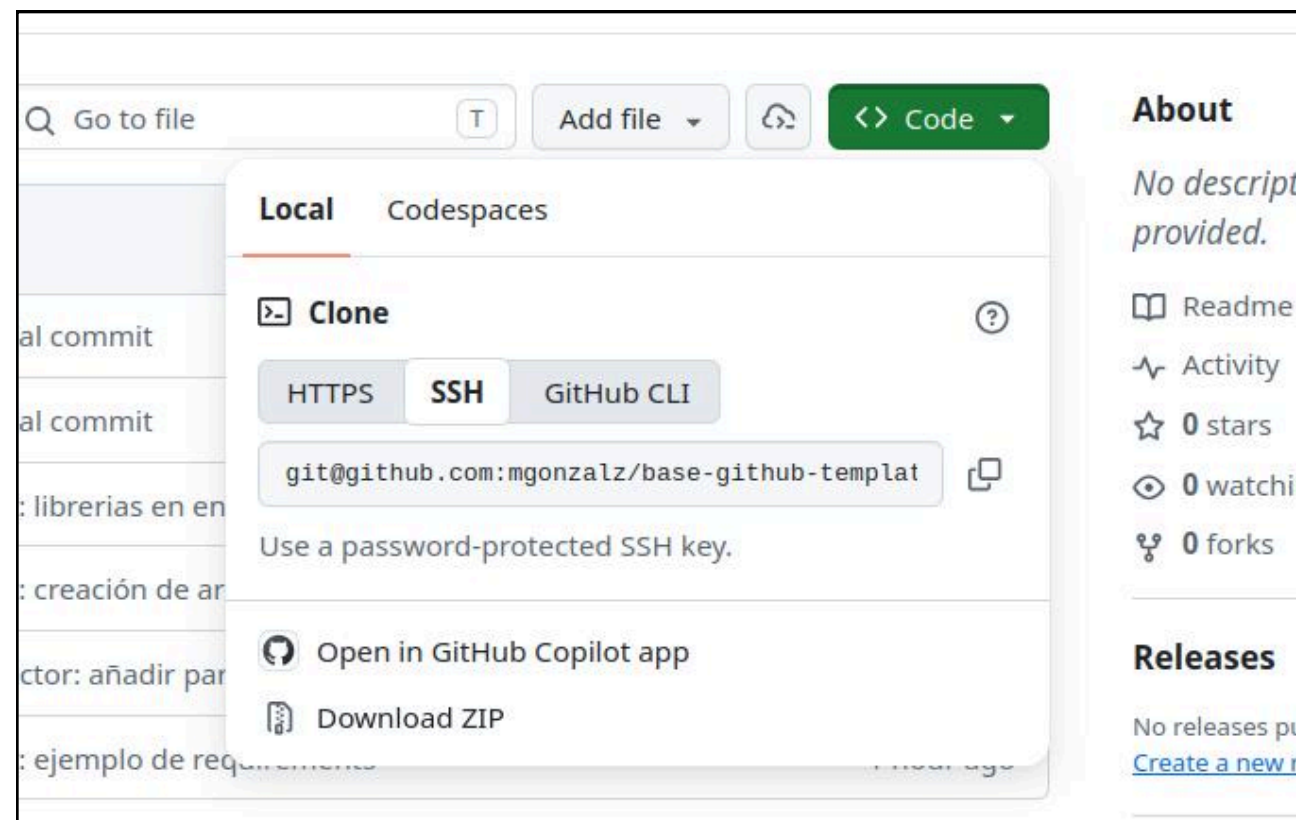
Git es la herramienta; GitHub, el sitio donde vive el repositorio

Repositorio

El proyecto con todo el historial de cambios

Clone

Descargar un repositorio por primera vez



Add / Commit

Una foto de los cambios, con mensaje, autor y fecha

- Los commits NO STAGED, no se suben

Push / Pull

Subir commits, traer los commits

Rama / Branch

Una línea de trabajo paralela; la principal es main

Anatomía de un repositorio

Lo que debería tener cualquier repositorio

- **README.md**: Qué es, cómo instalarlo, cómo ejecutarlo y qué resultados da
- **requirements.txt**
- **.gitignore**: Fuera entornos, datos, pesos y secretos
- **src/**: El código reutilizable; los notebooks, solo para explorar
- **tests/**: Comprobaciones pequeñas que corren en CPU
- **LICENSE**: Qué pueden hacer otros con tu código si es público
- **Workflows**: Tests y tareas automáticas cada vez que subes código.

```
practica-gpu/  
├── README.md  
├── LICENSE  
├── .gitignore  
├── requirements.txt # o pyproject.toml  
├── src/  
│   ├── model.py  
│   └── train.py  
├── notebooks/  
├── tests/  
│   └── test_model.py  
└── .github/workflows/
```

.gitignore

Lista de lo que nunca debe subirse a GitHub

Archivo que indica a Git qué archivos y carpetas debe ignorar y no incluir en el repositorio.

- Entornos no portables, archivos pesados (GitHub rechaza los de más de 100 MB), secretos y basura del sistema.

Al crear el repo en GitHub, elige la plantilla de Python.

**.gitignore no borra
lo ya versionado:
git rm --cached
archivo y commit.**

Great repository names are short and memorable. How

Description

0 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository

Start with a template
Templates pre-configure your repository with files.

Add README
READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)

Add license
Licenses explain how others can use your code. [About licenses](#)

Choose a .gitignore template

Search templates

- ✓ No .gitignore
- AL
- Actionscript
- Ada
- AdventureGameStudio
- Agda
- Android
- Angular
- AppEngine
- AppceleratorTitanium
- ArchLinuxPackages

No .gitignore

No license

base-github-template / .gitignore

mgonzalz Initial commit

Code Blame 218 lines (178 loc) · 4.52 KB

```

1 # Byte-compiled / optimized / DLL files
2 __pycache__/
3 *.py[cod]
4 *$py.class
5
6 # C extensions
7 *.so
8
9 # Distribution / packaging
10 .Python
11 build/
12 develop-eggs/
13 dist/
14 downloads/
15 eggs/
16 .eggs/
17 lib/
18 lib64/
19 parts/
20 sdist/
21 var/
22 wheels/
23 share/python-wheels/
24 *.egg-info/
25 .installed.cfg
26 *.egg
27 MANIFEST
28
29 # PyInstaller
30 #   Usually these files are written by a python script from a template
31 #   before PyInstaller builds the exe, so as to inject date/other infos into it.
32 *.manifest
33 *.spec
34
35 # Installer logs
36 pip-log.txt
37 pip-delete-this-directory.txt
38
39 # Unit test / coverage reports

```

GitHub Actions

Tests y tareas automáticas cada vez que se sube código

Qué es

Una máquina de GitHub ejecuta estos pasos en cada push o pull request.

Sin GPU AMD

Los tests deben poder correr en CPU: otra razón para el código agnóstico de dispositivo.

Más usos

Linters como ruff, generar documentación, publicar un paquete.

.github/workflows/tests.yml

```
name: tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-python@v6
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: pytest
```

Más de GitHub

Funcionalidades extra



Organizaciones

Agrupar repos, p. ej. los de la asignatura, con equipos y permisos



Issues y Projects

Organizar las tareas del grupo en un tablero



Git LFS

Versionar archivos grandes, como pesos de modelos



GitHub Pages

Una web gratuita desde un repo: portfolio o documentación



Codespaces

Un entorno de desarrollo en la nube desde el navegador



GitHub Education

Student Developer Pack: herramientas gratis para estudiantes

Autenticación

Cómo demostrarle a GitHub quienes somos sin dejar rastro en el lab

Clave SSH

Pega la clave pública (.pub) en Settings → SSH and GPG keys. Configurar una vez, olvidarse.



```
ssh-keygen -t ed25519 -C "tu@correodegithub.es"
```

Token (PAT) HTTPS

Settings → Developer settings → Personal access tokens. Hace de contraseña: GitHub ya no acepta la tuya en Git.

Mejor fine-grained, limitado a un repositorio y con caducidad corta.

En el PC del laboratorio: token temporal, sin guardar credenciales, y revócalo al terminar si sospechas que se ha quedado grabado.

The screenshot shows the GitHub 'New fine-grained personal access token' page. On the left, a sidebar contains links for 'GitHub Apps', 'OAuth Apps', 'Personal access tokens' (selected), 'Fine-grained tokens' (highlighted), and 'Tokens (classic)'. The main content area is titled 'New fine-grained personal access token' and includes the following sections:

- Create a fine-grained, repository-scoped token suitable for personal API use and for using Git over HTTPS.**
- Token name ***: A text input field with a placeholder.
- Description**: A large text area for describing the token.
- Resource owner**: A dropdown menu showing 'mgonzalz'.
- Expiration**: A date picker set to '30 days (Oct 23, 2026)'.
- Repository access**: Radio buttons for 'Public repositories' (selected), 'All repositories', and 'Only select repositories'.

¡Gracias por su Atención!

Soy Innovación

Soy Maker

Soy UAX